The Project, Explained in Plain Language

A no-jargon guide so you can confidently explain this work to a professor — even with no cybersecurity background. Read it once; the last page is a cheat sheet for likely questions.

The whole thing in one breath: When a new software flaw is announced publicly, nobody knows how urgent it is yet. This project built a tool that, on day one, predicts *how soon* that flaw is likely to be attacked — and at that critical first moment it does a measurably better job than the industry-standard scoring system used today.

First, the 8 words you need

If you understand these eight terms, you understand the whole project. Don't memorize definitions — remember the analogy in the right-hand column.

Term	What it really means	Everyday analogy
CVE	A public ID number for one known software flaw (e.g. CVE-2024-1234).	A recall notice for a car defect — every flaw gets a catalog number.
Vulnerability	A weakness in software that someone could abuse.	An unlocked window in a house.
Exploit	Actual code or a technique that abuses the weakness.	The burglar's tool kit made for that exact window.
"In the wild"	The flaw is being attacked for real, not just in theory.	A confirmed break-in — not a security demo.
PoC	"Proof of concept" — a demo showing the flaw *can* be abused.	A locksmith proving the lock can be picked. No theft yet.
EPSS	An industry score (0–100%): "will this be attacked in the next 30 days?"	A weather forecast: "30% chance of rain."
KEV	An official US-government list of flaws confirmed to be attacked.	A police blotter of confirmed crimes.
Survival analysis	Math for predicting *how long until* an event happens — even for cases where it hasn't yet.	How doctors predict "time until relapse."

The problem this project tackles

Every day, hundreds of new software flaws (CVEs) are published. Defenders — the people who keep companies safe — cannot fix all of them at once. They must guess which few will actually be attacked, and how soon. Today the main tool for this, **EPSS**, answers a yes/no question: "will this flaw be attacked in the next 30 days?" This project asks a richer question: **WHEN will it be attacked — days, weeks, or months from now?**

Think of it like this. EPSS is like a smoke alarm: it tells you *whether* there's a fire right now. This project is more like a *weather forecast* that says the storm will probably hit Thursday afternoon. Knowing the *timing* lets defenders line up and fix the most urgent flaws first.

What was actually built

- A prediction tool that, the moment a flaw is announced, estimates how soon it is likely to be weaponized — using only facts known on day one (how severe it is, what type of weakness it is, which vendor made the software, and so on).
- It learned from a large history of about **338,000 past flaws** and what happened to each one.
- It uses **survival analysis** — the same kind of math doctors use to predict "time until a patient relapses." Here the "patient" is a software flaw and the "relapse" is the first attack.

The main result, in plain English

To grade a tool that *ranks* things by risk, researchers use a number called **AUC**. It runs from **0.5** (a coin flip — useless) to **1.0** (perfect). Our tool scores about **0.80**, which is good.

We put our tool head-to-head against EPSS on the exact same task. Our tool scored **+0.10 higher** (0.80 versus EPSS's 0.70). The gap is biggest at **"day zero"** — the moment a flaw is first announced, when EPSS has barely any information to work with. That is exactly the moment a defender most needs a good guess.

Think of it like this. Imagine 100 brand-new flaws announced today, and only a few will ever be attacked. Line them up from "most likely to be attacked" to "least." Our tool puts the truly dangerous ones nearer the top of that list than EPSS does — on the very first day, before anyone else knows which is which.

Why a professor should trust the result

Good science is mostly about being honest, especially about your own work. Two things here show that honesty — and they are the strongest points to make:

- **We found and fixed a mistake that was making us look TOO good.** By accident, EPSS was being run through an extra step that crippled it down to 0.50 (a coin flip), which would have made our win look enormous. We corrected the comparison so EPSS is judged *fairly* — and we still win, just by a sensible, believable amount.
- **We are upfront that our "real attack" data is a stand-in.** Perfect records of exactly when each flaw was first attacked don't fully exist. We use the best available list, which tends to lag reality by about **6 months**. So we describe the result carefully: "better at ranking at the start," not "we perfectly predict real attacks."
- **An independent double-check.** A separate adversarial review was run specifically to look for cheating or hidden shortcuts in the result. It found none. (We also checked the report's academic references and quietly fixed three small errors in them.)

What comes next

- Try a new live data source (called GreyNoise) that watches real internet attacks as they happen. Catch: it only sees attacks *going forward*, not in the past, so it helps future predictions, not the historical test.
- Agree on the honest headline for the thesis: "a better early-warning ranking than the industry standard," while being clear about the 6-month data limitation.

If the professor asks… (cheat sheet)

Q: "Isn't this just EPSS again?"
A: No. EPSS gives a yes/no 30-day probability. Ours predicts the *timing* and works noticeably better at the very first moment a flaw is announced, when EPSS has little to go on.

Q: "How much better is it, really?"
A: About +0.10 on a 0.5-to-1.0 scale (0.80 vs 0.70), and the advantage is largest at day zero. Modest but real, and carefully verified.

Q: "How do you know it isn't a fluke or cheating?"
A: It was tested across 15 separate time periods, always training on the past and predicting the future — never peeking ahead. An independent adversarial review also looked for shortcuts and found none.

Q: "What's the catch?"
A: True 'real-world attack' data is scarce, so we use a stand-in list that lags by about 6 months. The honest claim is an early-warning *ranking* advantage, and the real limit is the *data*, not the method.

Q: "Why does this matter?"
A: Defenders can't fix everything at once. Pointing to the flaws most likely to be attacked *soon* — right when they appear — saves time and reduces risk.

Q: "Why this kind of model and not fancy AI?"
A: They tested many models, including deep-learning ones. With this little confirmed-attack data, the simpler, classical survival model did just as well or better. More data — not a fancier model — is what would

help.

Your one-sentence pitch: “We built a tool that, the moment a software flaw is announced, predicts how soon it’s likely to be attacked — and at that critical first moment it beats the industry-standard forecast, which we proved carefully and honestly.”